

# RAPPORT TECHNIQUE

## UniPlan

### Application Web de planification intelligente des examens

**Réalisé par :**

Safia Massaid

#### **Technologies utilisées :**

- Python (Flask)
- PostgreSQL
- HTML / CSS / JavaScript

**Date: Mars 2026**

#### **Résumé**

UniPlan est une application web permettant d'automatiser la planification des examens en respectant les contraintes académiques, organisationnelles et logistiques.

Le système repose sur une approche combinatoire afin de générer des plannings optimisés, fiables et sans conflits.

## **Introduction Générale**

La planification des examens de session constitue une étape essentielle et stratégique dans le fonctionnement d'un établissement universitaire. Elle ne se limite pas à une simple organisation temporelle, mais représente un véritable problème d'optimisation impliquant la coordination simultanée de plusieurs ressources humaines et matérielles. En effet, il s'agit d'attribuer de manière cohérente et efficace des créneaux horaires, des salles ainsi que des enseignants surveillants à un ensemble d'épreuves, tout en respectant un ensemble complexe de contraintes pédagogiques et organisationnelles.

Dans un contexte universitaire marqué par l'augmentation du nombre d'étudiants, la diversification des filières et la limitation des infrastructures disponibles, la planification manuelle devient rapidement une tâche fastidieuse, chronophage et sujette à de nombreuses erreurs. Parmi les problèmes fréquemment rencontrés figurent les conflits d'horaires, la double affectation des salles ou des enseignants, ainsi que le non-respect des capacités d'accueil des locaux. Ces dysfonctionnements peuvent engendrer des perturbations importantes dans le déroulement des examens et nuire à la qualité de l'organisation académique.

Face à ces enjeux, l'automatisation du processus de planification apparaît comme une solution pertinente et nécessaire. Dans ce cadre, le présent projet propose le développement d'une application web dédiée à la génération automatique des plannings d'examens. Cette application repose sur une approche d'optimisation combinatoire, permettant de modéliser le problème de manière formelle et de rechercher des solutions optimales ou quasi-optimales en tenant compte des différentes contraintes.

L'implémentation de cette solution est réalisée en langage Python, en s'appuyant sur le framework Flask pour la conception de l'interface web. L'objectif principal est de fournir un outil performant, flexible et ergonomique, capable d'assister les responsables de scolarité dans la gestion des examens, tout en garantissant la validité et l'équité des affectations. En outre, l'application vise à améliorer la fiabilité du processus, à réduire les risques d'erreurs humaines et à faciliter la prise de décision grâce à une visualisation claire des plannings générés.

## **Chapitre 1 : Contexte et État de l'Art**

### **1.1 Introduction**

Dans les établissements universitaires modernes, la gestion des examens constitue une activité fondamentale qui mobilise des ressources importantes, tant humaines que matérielles. Avec l'augmentation continue du nombre d'étudiants, la diversification

des parcours académiques et la multiplication des modules, l'organisation des sessions d'examens devient de plus en plus complexe.

Traditionnellement, cette planification est réalisée de manière manuelle ou semi-manuelle, souvent à l'aide de tableaux ou de fichiers bureautiques. Toutefois, cette approche présente plusieurs limites majeures : elle est chronophage, difficile à maintenir et fortement sujette aux erreurs humaines. Parmi les problèmes fréquemment rencontrés figurent les conflits d'horaires pour les étudiants, la double affectation des enseignants ou encore une mauvaise gestion des capacités des salles.

Dans ce contexte, le recours à des outils informatiques intelligents apparaît comme une nécessité. Ces outils permettent non seulement d'automatiser la génération des plannings, mais également d'optimiser l'utilisation des ressources disponibles tout en respectant un ensemble de contraintes complexes.

Ce chapitre a pour objectif de présenter le contexte général du problème, de définir formellement la planification des examens, puis d'explorer les principales approches proposées dans la littérature pour sa résolution.

## 1.2 Problème de planification (Scheduling)

### 1.2.1 Définition générale

Le problème de planification, également appelé *scheduling*, est un problème classique en informatique et en recherche opérationnelle. Il consiste à affecter un ensemble de ressources limitées à un ensemble de tâches, tout en respectant un ensemble de contraintes.

Dans un cadre formel, un problème de planification peut être défini par :

- un ensemble de **tâches** (ici, les examens),
- un ensemble de **ressources** (salles, enseignants, créneaux),
- un ensemble de **contraintes** (temps, capacité, disponibilité),
- et éventuellement un **objectif d'optimisation** (minimisation des conflits, équilibrage des charges, etc.).

Dans le contexte universitaire, ce problème consiste à attribuer à chaque examen :

- un créneau horaire,
- une ou plusieurs salles,
- un ou plusieurs enseignants pour la surveillance,

tout en garantissant l'absence de conflits pour les étudiants et le personnel.

### 1.2.2 Spécificité de la planification des examens

La planification des examens se distingue des autres problèmes de planification par plusieurs contraintes spécifiques liées au domaine académique.

Parmi les principales contraintes, on retrouve :

- **Conflits étudiants :**

Un étudiant ne peut pas passer deux examens simultanément. Cette contrainte est critique et doit être strictement respectée.

- **Disponibilité des enseignants :**

Un enseignant ne peut être affecté qu'à une seule surveillance à un moment donné.

- **Contraintes de capacité :**

Chaque salle possède une capacité maximale qui ne doit pas être dépassée.

- **Synchronisation des groupes :**

Un même module doit généralement être examiné au même moment pour tous les groupes concernés.

- **Équité organisationnelle :**

Il est souhaitable de répartir équitablement les surveillances entre les enseignants et d'éviter la surcharge sur une même journée.

- **Contraintes logistiques :**

Limiter les déplacements inutiles des enseignants entre les salles et optimiser l'utilisation des infrastructures.

Ces contraintes rendent le problème particulièrement complexe et difficile à résoudre manuellement.

## 1.3 État de l'art : approches de résolution

La planification des examens est largement étudiée dans la littérature scientifique et est classée comme un problème **NP-difficile**, ce qui signifie qu'il n'existe pas de solution exacte efficace pour les grandes instances.

Plusieurs approches ont été proposées :

### 1.3.1 Coloration de graphes

Cette approche consiste à modéliser les examens sous forme de graphe :

- chaque nœud représente un examen,
- une arête relie deux examens s'ils partagent des étudiants.

L'objectif est d'attribuer une "couleur" (créneau) à chaque nœud sans que deux nœuds adjacents aient la même couleur.

Avantage : simple et intuitive

Limite : ne prend pas facilement en compte toutes les contraintes réelles

### 1.3.2 Métaheuristiques

Les métaheuristiques sont des méthodes d'optimisation approchée, parmi lesquelles :

- algorithmes génétiques
- recuit simulé
- recherche tabou

Ces méthodes explorent l'espace des solutions pour trouver une solution optimale ou quasi-optimale.

Avantage : efficaces pour les grands problèmes

Limite : ne garantissent pas toujours une solution parfaite

### 1.3.3 Programmation par contraintes (CSP)

Dans cette approche, le problème est modélisé à l'aide de variables, de domaines et de contraintes.

Le solveur cherche une solution satisfaisant toutes les contraintes.

Avantage : garantit des solutions valides

Limite : peut être coûteux en temps de calcul

### 1.3.4 Approches combinatoires et logiques

Ces méthodes reposent sur :

- des variables booléennes
- des expressions logiques
- des règles de décision

Elles permettent une modélisation fine et flexible du problème.

Avantage : grande précision dans la modélisation

Limite : complexité de mise en œuvre

### 1.3.5 Approche retenue dans ce projet

Dans ce projet, nous adoptons une **approche combinatoire** basée sur :

- des variables logiques,
- un ensemble de contraintes fortes et souples,
- un mécanisme d'optimisation,

le tout implémenté en Python avec une interface web développée à l'aide du framework Flask.

## 1.4 Conclusion du chapitre

La planification des examens constitue un problème complexe relevant de l'optimisation combinatoire. Les différentes approches étudiées montrent qu'aucune méthode unique ne permet de résoudre parfaitement toutes les instances, ce qui justifie le recours à des solutions hybrides et adaptées au contexte.

Dans ce travail, nous proposons une solution combinant modélisation formelle et implémentation pratique, afin de répondre aux besoins réels d'un établissement universitaire.

Le chapitre suivant sera consacré à la **modélisation détaillée du problème** ainsi qu'à la définition des contraintes et des variables utilisées.

## **Chapitre 2 : Analyse et Spécification des Besoins**

### **2.1 Introduction**

Après avoir présenté le contexte général et les différentes approches de résolution du problème de planification des examens, il est essentiel de procéder à une analyse approfondie des besoins du système à concevoir. Cette étape constitue un pilier fondamental dans le cycle de développement, car elle permet de définir précisément les attentes des utilisateurs ainsi que les contraintes auxquelles le système devra répondre.

L'objectif de ce chapitre est de formaliser les besoins fonctionnels et non fonctionnels du système, d'identifier les ressources impliquées dans le processus de planification, et de caractériser les différentes contraintes du domaine. Cette analyse permet également de délimiter le périmètre du projet et de poser les bases nécessaires à la phase de conception.

### **2.2 Étude de l'existant**

Dans de nombreux établissements universitaires, la planification des examens repose encore sur des outils traditionnels tels que les tableurs (Excel) ou des documents bureautiques. Bien que ces outils offrent une certaine flexibilité, ils présentent plusieurs limites lorsqu'il s'agit de gérer des volumes importants de données et de contraintes.

En effet, la gestion manuelle du planning implique :

- une manipulation complexe de plusieurs fichiers,
- une forte dépendance à l'intervention humaine,
- une difficulté à vérifier automatiquement les contraintes.

Cette approche entraîne souvent :

- des conflits d'horaires pour les étudiants,
- des chevauchements dans l'affectation des enseignants,
- une mauvaise exploitation des capacités des salles,
- et une perte de temps considérable lors des phases de vérification et de correction.

De plus, l'absence d'automatisation rend difficile l'optimisation du planning, notamment en ce qui concerne l'équilibrage des charges ou la réduction des périodes de forte concentration d'examens.

Face à ces limitations, il devient nécessaire de mettre en place un système informatisé capable de :

- centraliser les données,
- automatiser la génération des plannings,
- détecter et éviter les conflits,
- et proposer des solutions optimisées.

## **2.3 Contraintes du domaine**

La planification des examens est un problème fortement contraint. Une bonne compréhension de ces contraintes est indispensable pour garantir la validité des solutions générées.

### **2.3.1 Identification des ressources**

Le système repose sur plusieurs types de ressources, chacune possédant ses propres caractéristiques :

## **1. Les salles**

- Capacité maximale (nombre d'étudiants)
- Type de salle (classe, amphithéâtre)
- Disponibilité selon les créneaux

## **2. Les enseignants**

- Spécialité ou module enseigné
- Disponibilité
- Charge maximale de surveillance

## **3. Les modules (examens)**

- Niveau (L1, L2, Master...)
- Filière (informatique, mathématiques...)
- Nombre d'étudiants inscrits
- Durée de l'examen

## **4. Les créneaux horaires**

- Jours de la session
- Périodes (matin / après-midi)
- Durée des créneaux

### **2.3.2 Contraintes fortes (Hard Constraints)**

Les contraintes fortes sont des règles strictes qui doivent impérativement être respectées. Toute violation rend la solution invalide.

- Un étudiant ne peut pas passer deux examens au même moment.
- Un enseignant ne peut pas surveiller deux examens simultanément.
- Une salle ne peut pas accueillir un nombre d'étudiants supérieur à sa capacité.
- Chaque examen doit être affecté à un créneau valide.
- Une salle ne peut accueillir qu'un seul examen par créneau.
- Tous les groupes d'un même module doivent passer l'examen au même moment (selon le cas).

### **2.3.3 Contraintes souples (Soft Constraints)**

Les contraintes souples représentent des préférences. Leur non-respect n'invalide pas la solution, mais diminue sa qualité.

- Éviter que les étudiants aient plusieurs examens rapprochés.
- Répartir les examens de manière équilibrée sur la période.
- Éviter la surcharge d'un enseignant sur une même journée.
- Minimiser les déplacements des enseignants entre les salles.
- Éviter les créneaux surchargés en termes de ressources utilisées.

Ces contraintes peuvent être intégrées sous forme de fonction de coût à minimiser.

## **2.4 Spécification fonctionnelle et non fonctionnelle**

### **2.4.1 Besoins fonctionnels**

Le système doit offrir un ensemble de fonctionnalités permettant une gestion complète du processus de planification :

#### ***Gestion des données***

- Saisie manuelle des données (modules, salles, enseignants, étudiants)
- Importation des données via fichiers (CSV, Excel)
- Modification et suppression des données

#### ***Configuration***

- Définition des créneaux horaires
- Paramétrage des contraintes (fortes et souples)
- Définition des règles d'optimisation

#### ***Génération du planning***

- Lancement de l'algorithme de planification
- Vérification automatique des contraintes
- Génération d'un planning valide

### **Visualisation**

- Affichage du planning sous forme de tableau
- Visualisation par jour, par salle ou par module
- Mise en évidence des conflits éventuels

### **Modification**

- Ajustement manuel des affectations
- Recalcul partiel du planning

### **Exportation**

- Export du planning en PDF
- Export en Excel pour exploitation externe

## **2.4.2 Besoins non fonctionnels**

Les besoins non fonctionnels définissent les exigences de qualité du système :

### **Performance**

- Temps de génération acceptable même pour de grandes instances
- Optimisation des ressources système

### **Ergonomie**

- Interface web simple, intuitive et accessible
- Navigation fluide entre les différentes fonctionnalités

### **Fiabilité**

- Respect strict des contraintes fortes
- Robustesse face aux erreurs de saisie

### **Sécurité**

- Authentification des utilisateurs
- Gestion des rôles (administrateur, responsable, consultant)

- Protection des données sensibles

### ***Maintenabilité***

- Code structuré et modulaire
- Facilité d'évolution du système

## **2.6 Conclusion du chapitre**

L'analyse des besoins met en évidence la complexité du processus de planification des examens, ainsi que la nécessité d'un système capable de gérer simultanément un grand nombre de contraintes et de ressources.

Cette étape a permis de définir clairement :

- les fonctionnalités attendues du système,
- les contraintes à respecter,
- les exigences de qualité.

Ces éléments serviront de base pour la phase suivante, consacrée à la conception et à la modélisation détaillée du système.

## **Chapitre 3 : Conception et Modélisation du Système**

### **3.1 Introduction**

Après l'analyse des besoins présentée dans le chapitre précédent, cette partie est consacrée à la conception du système proposé. Cette étape joue un rôle essentiel dans le cycle de développement, car elle permet de traduire les besoins fonctionnels et non fonctionnels en une architecture technique et logique cohérente.

L'objectif de ce chapitre est de décrire :

- l'architecture globale du système,
- les choix technologiques adoptés,
- la modélisation des données et des traitements,
- ainsi que la représentation UML des différents composants.

Cette phase permet d'assurer une vision claire et structurée du système avant son implémentation.

## 3.2 Architecture globale du système

Le système développé repose sur une architecture **client-serveur**, largement utilisée dans les applications web modernes. Cette architecture permet de séparer les responsabilités entre l'interface utilisateur et la logique métier, garantissant ainsi une meilleure modularité et maintenabilité.

### 3.2.1 Vue générale de l'architecture

Le système se compose de deux parties principales :

#### 1. Le client (Front-end)

Le client correspond au navigateur web utilisé par l'utilisateur. Il permet :

- la saisie des données via des formulaires,
- la visualisation du planning,
- l'interaction avec le système.

Les technologies utilisées incluent :

- HTML pour la structure,
- CSS pour le style,
- JavaScript pour l'interactivité.

#### 2. Le serveur (Back-end)

Le serveur est développé en Python en utilisant le framework Flask. Il assure :

- la gestion de la logique métier,
- le traitement des données,

- l'exécution de l'algorithme de planification,
- la communication avec la base de données.

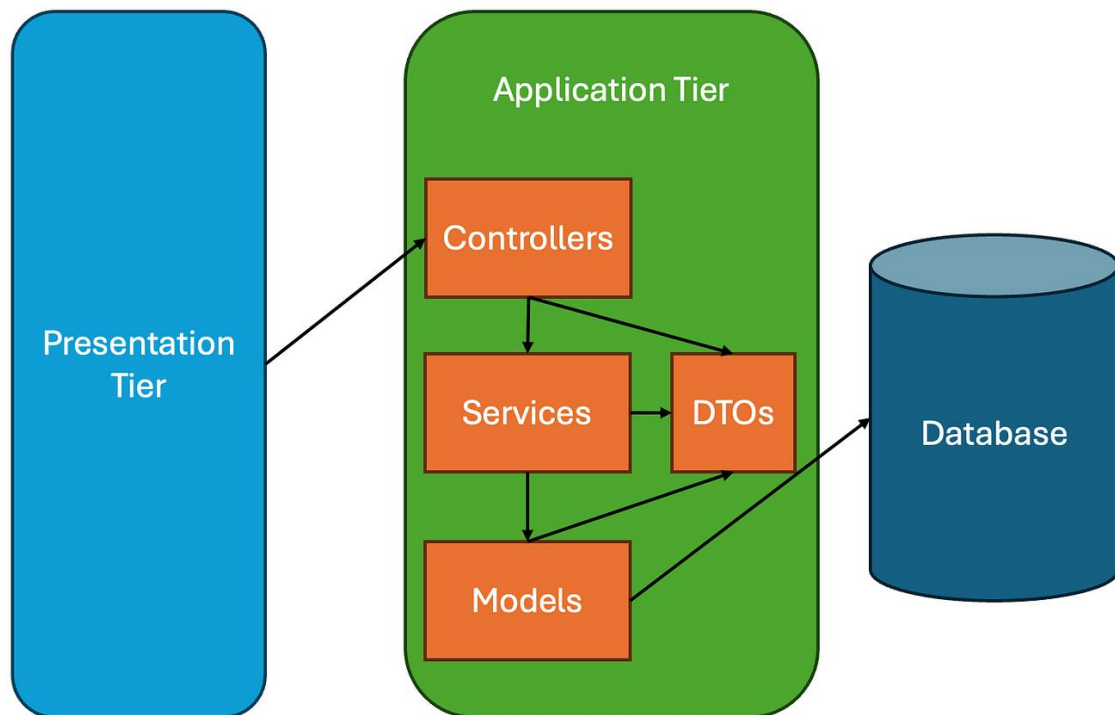


Figure 3.1 – Architecture globale du système de planification

### 3.2.2 Composants du système

Le système est organisé en plusieurs modules :

#### ◇ Module de gestion des données

- Gestion des salles, enseignants, modules, étudiants
- CRUD (Create, Read, Update, Delete)

#### ◇ Module de modélisation combinatoire

- Définition des variables booléennes
- Construction des contraintes logiques

#### ◇ Module d'algorithme

- Implémentation du backtracking
- Gestion des conflits

- Optimisation

#### ◊ *Module interface utilisateur*

- Affichage du planning
- Interaction avec l'utilisateur

### 3.3 Modélisation logique et traitement des conflits

#### 3.3.1 Formalisation des états (variables booléennes)

Pour modéliser le problème de planification, nous utilisons une approche basée sur des variables booléennes.

Chaque examen est associé à une variable :

- $(x_{\{i,j\}} = 1)$  : l'examen  $i$  est programmé au créneau  $j$
- $(x_{\{i,j\}} = 0)$  : sinon

Cette représentation permet de transformer le problème en un système logique où chaque contrainte est exprimée sous forme d'équations booléennes.

Par exemple :

- Un examen doit être affecté à un seul créneau
- Deux examens en conflit ne peuvent pas partager le même créneau

Ces contraintes sont exprimées à l'aide des opérateurs :

- ET ( $\wedge$ )
- OU ( $\vee$ )
- NON ( $\neg$ )

#### 3.3.3 Algorithme d'affectation

L'algorithme proposé repose sur une approche combinatoire utilisant le **backtracking** guidé par les contraintes.

##### **Étapes principales :**

1. Lecture des données (modules, salles, enseignants)
2. Génération des créneaux possibles
3. Construction des contraintes logiques

4. Recherche d'une solution valide via backtracking
5. Optimisation selon les contraintes souples

### **Principe :**

- L'algorithme explore les différentes combinaisons possibles
- Il élimine immédiatement les solutions invalides
- Il converge vers une solution respectant toutes les contraintes

## Algorithme — Moteur de Planification Examens & Surveillance

### Fonctions Utilitaires

```
FONCTION time_to_minutes(t_str)
  h, m ← DÉCOUPER t_str par ':'
  RETOURNER h × 60 + m
FONCTION minutes_to_time(mins)
  RETOURNER FORMAT("%02d:%02d", mins ÷ 60, mins MOD 60)
```

### 1. Génération des créneaux d'un jour

```
FONCTION generate_slots_for_day(jour, durée_examen=90, pause=15)
  slots ← liste vide

  // — Créneaux du MATIN —
  curseur ← time_to_minutes(jour.heure_debut_matin)
  fin_matin ← time_to_minutes(jour.heure_fin_matin)

  TANT QUE curseur + durée_examen ≤ fin_matin FAIRE
    AJOUTER (minutes_to_time(curseur), minutes_to_time(curseur + durée_examen)) à
slots
    curseur ← curseur + durée_examen + pause

  // — Créneaux de l'APRÈS-MIDI —
  curseur ← time_to_minutes(jour.heure_debut_aprem)
  fin_aprem ← time_to_minutes(jour.heure_fin_aprem)

  TANT QUE curseur + durée_examen ≤ fin_aprem FAIRE
    AJOUTER (minutes_to_time(curseur), minutes_to_time(curseur + durée_examen)) à
slots
    curseur ← curseur + durée_examen + pause
```

RETOURNER slots

## 2. Filtrer les salles disponibles à une date

FONCTION get\_salles\_disponibles(toutes\_salles, date\_examen)  
disponibles  $\leftarrow$  liste vide

POUR CHAQUE salle DANS toutes\_salles FAIRE  
  est\_libre  $\leftarrow$  VRAI  
  POUR CHAQUE indispo DANS salle.indisponibilites FAIRE  
    date\_fin  $\leftarrow$  indispo.date\_fin OU indispo.date\_debut  
    SI indispo.date\_debut  $\leq$  date\_examen  $\leq$  date\_fin ALORS  
      est\_libre  $\leftarrow$  FAUX  
      SORTIR de la boucle  
  SI est\_libre ALORS  
    AJOUTER salle à disponibles  
RETOURNER disponibles

## 3. Calculer les salles nécessaires pour N étudiants

FONCTION compute\_salles\_for\_students(total\_étudiants, salles\_dispo)  
amphis  $\leftarrow$  TRIER [s  $\in$  salles\_dispo | s.type = 'amphi'] par capacité DESC  
classes  $\leftarrow$  TRIER [s  $\in$  salles\_dispo | s.type = 'salle'] par capacité DESC

nécessaires  $\leftarrow$  liste vide  
restant  $\leftarrow$  total\_étudiants

POUR CHAQUE salle DANS (amphis + classes) FAIRE  
  SI restant  $\leq$  0 ALORS SORTIR  
  AJOUTER salle à nécessaires  
  restant  $\leftarrow$  restant - salle.capacité

RETOURNER nécessaires, (restant  $\leq$  0)

#### 4. Calculer le nombre d'étudiants pour une matière

```
FONCTION compute_students_for_matiere(matière)
niveau ← matière.niveau
spécialité ← matière.spécialité

SI spécialité EXISTE ALORS
    sections ← CHERCHER sections OÙ id_niveau = niveau.id
    total ← SOMME(groupe.nb_etudiant POUR section, groupe)
    nb_spécialités ← NOMBRE(niveau.spécialités) OU 1
    SI nb_spécialités > 1 ALORS
        total ← total ÷ nb_spécialités
    RETOURNER total
SINON
    // Tronc commun → tous les étudiants
    sections ← CHERCHER sections OÙ id_niveau = niveau.id
    RETOURNER SOMME(groupe.nb_etudiant POUR section, groupe)
```

#### 5. ALGORITHME PRINCIPAL — generate\_planning

```
FONCTION generate_planning(id_session, durée_examen=90, pause=15)

// =====
// ÉTAPE 0 : Validation des données
// =====

session ← CHERCHER SessionExam(id_session)
SI session N'EXISTE PAS → ERREUR "Session introuvable"

jours ← CHERCHER JourExam OÙ id_session, TRIÉ par date
SI jours EST VIDE → ERREUR "Aucun jour"

toutes_salles ← CHERCHER toutes les Salle
tous_enseignants ← CHERCHER tous les Teacher
SI toutes_salles VIDE → ERREUR
SI tous_enseignants VIDE → ERREUR

// Créer la session de planning (brouillon)
planning_session ← NOUVEAU PlanningSession(statut='brouillon')
SAUVEGARDER planning_session
```

```
erreurs ← liste vide  
tous_pe ← liste vide
```

```
// Pré-calcul : disponibilités enseignants  
dispo_par_date ← dictionnaire { (teacher_id, date) → ensemble de (h_début, h_fin) }  
POUR CHAQUE enseignant, POUR CHAQUE dispo → REMPLIR dispo_par_date
```

```
// Structures de suivi des conflits  
usage_salles ← dictionnaire { date → { slot → ensemble de salle_ids } }  
usage_niveaux ← dictionnaire { date → { slot → ensemble de "niveau-spécialité" } }  
charges_enseignants ← dictionnaire { teacher_id → nombre de surveillances }
```

```
// =====
```

```
// PHASE 1 : Placement des examens
```

```
// =====
```

```
POUR CHAQUE jour DANS jours FAIRE
```

```
    matières_du_jour ← CHERCHER JourExamMatiere OÙ id_jour  
    SI matières_du_jour VIDE → CONTINUER
```

```
    slots ← generate_slots_for_day(jour, durée_examen, pause)  
    SI slots VIDE → AJOUTER erreur, CONTINUER
```

```
    salles_dispo ← get_salles_disponibles(toutes_salles, jour.date)  
    SI salles_dispo VIDE → AJOUTER erreur, CONTINUER
```

```
    // Créer les objets CreneauExam en BDD  
    objets_slots ← liste vide  
    POUR CHAQUE (h_deb, h_fin) DANS slots FAIRE  
        créer CreneauExam → AJOUTER à objets_slots
```

```
    // Préparer la liste des examens à placer  
    examens_à_placer ← liste vide  
    POUR CHAQUE matière_du_jour FAIRE  
        nb_étudiants ← compute_students_for_matiere(matière)  
        SI nb_étudiants = 0 → AJOUTER erreur, CONTINUER  
        Créer ou récupérer l'objet Examen  
        AJOUTER {examen, matière, niveau, spécialité, nb_étudiants}
```

```
    // Trier par nombre d'étudiants DÉCROISSANT (plus contraint d'abord)  
    TRIER examens_à_placer par nb_étudiants DESC
```

```

// — Placement combinatoire —
POUR CHAQUE exam DANS examens_à_placer FAIRE
    placé ← FAUX

    POUR CHAQUE slot DANS objets_slots FAIRE
        clé_slot ← (slot.h_début, slot.h_fin)
        clé_conflit ← "niveau_id-spécialité_id"

        // Vérif 1 : Pas de conflit de niveau/spécialité sur ce créneau
        SI clé_conflit ∈ usage_niveaux[date][clé_slot] ALORS
            CONTINUER au slot suivant

        // Vérif 2 : Trouver les salles encore libres sur ce créneau
        salles_libres ← [s ∈ salles_dispo | s.id ∉ usage_salles[date][clé_slot]]
        SI salles_libres VIDE → CONTINUER

        // Vérif 3 : Assez de capacité ?
        salles_nécessaires, faisable ← compute_salles_for_students(nb_étu,
salles_libres)
        SI NON faisable → CONTINUER

        // PLACER L'EXAMEN
        restant ← nb_étudiants
        POUR CHAQUE salle DANS salles_nécessaires FAIRE
            nb_affecté ← MIN(restant, salle.capacité)
            pe ← CRÉER PlanningExam(session, examen, salle, slot, nb_affecté)
            AJOUTER pe à tous_pe
            restant ← restant - nb_affecté

        // Mettre à jour les structures de suivi
        usage_salles[date][clé_slot] ← usage_salles ∪ {salles utilisées}
        usage_niveaux[date][clé_slot] ← usage_niveaux ∪ {clé_conflit}
        LIER matière au créneau (CreneauMatiere)

        placé ← VRAI
        SORTIR de la boucle des slots

    SI NON placé → AJOUTER erreur "Impossible de placer ..."

SI erreurs NON VIDE → ROLLBACK, RETOURNER (FAUX, erreurs)

```

```

// =====
// PHASE 2 : Affectation de la surveillance
// =====
créneaux_utilisés_par_prof ← dictionnaire { teacher_id → ensemble de (date, slot) }
jours_par_prof ← dictionnaire { teacher_id → { date → compteur } }

POUR CHAQUE pe_data DANS tous_pe FAIRE
  date ← pe_data.date
  clé_slot ← pe_data.slot

  // Filtrer les enseignants disponibles
  candidats ← liste vide
  POUR CHAQUE enseignant DANS tous_enseignants FAIRE
    dispos ← dispo_par_date[(enseignant.id, date)]

    SI dispos EST VIDE ET aucune dispo déclarée → disponible
    SINON vérifier qu'un créneau dispo COUVRE le slot

    SI NON disponible → CONTINUER
    SI (date, clé_slot) ∈ créneaux_utilisés_par_prof[enseignant.id] → CONTINUER

    AJOUTER enseignant à candidats

  // Trier par : (1) nb total de surveillances, (2) nb surveillances ce jour
  TRIER candidats par (charges[id], jours_par_prof[id][date]) ASC

  SI candidats NON VIDE ALORS
    enseignant ← candidats[0] // le moins chargé
    CRÉER PlanningSurveillance(pe, enseignant, 'surveillant')
    charges_enseignants[enseignant.id] += 1
    créneaux_utilisés_par_prof[enseignant.id] ← U {(date, clé_slot)}
    jours_par_prof[enseignant.id][date] += 1

COMMIT en base de données

// =====
// PHASE 3 : Retour des statistiques
// =====
RETOURNER (VRAI, {
  session_id,

```

```

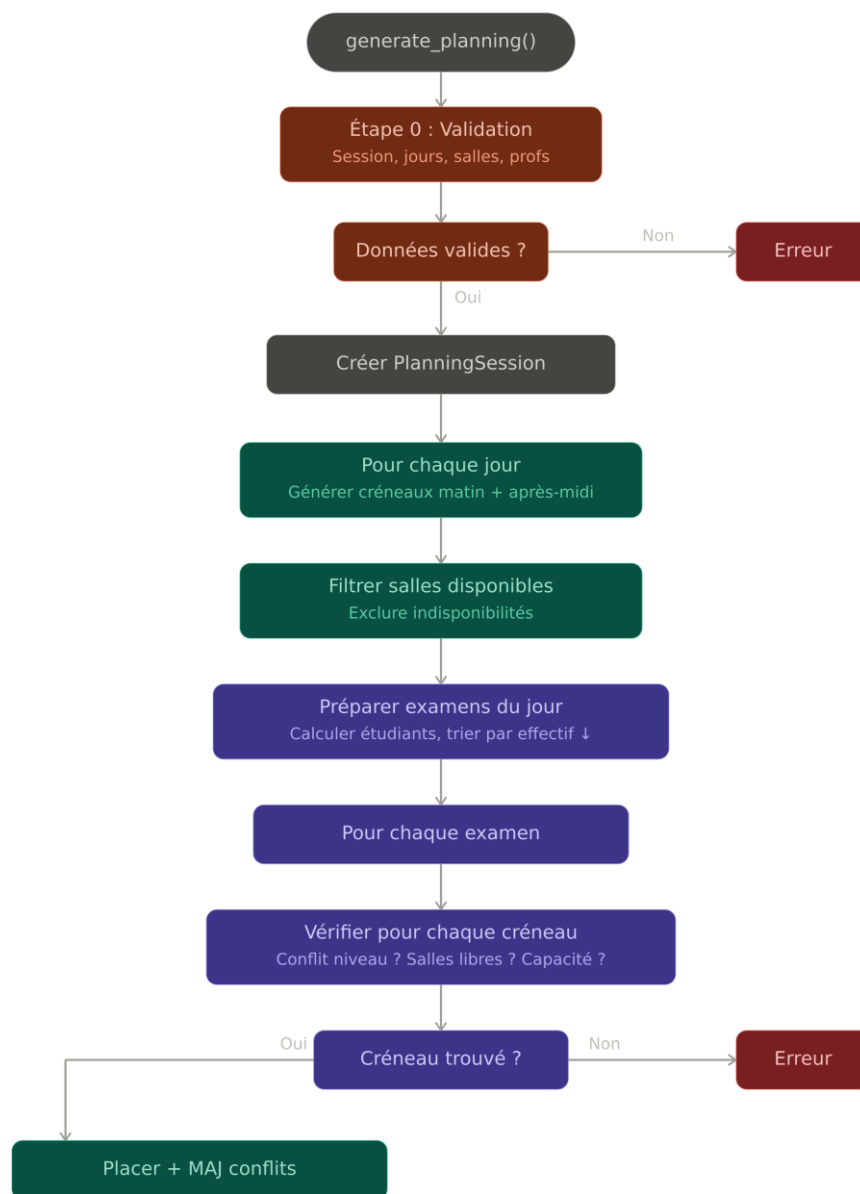
total_examens,
total_surveillances,
distribution_enseignants,
jours_traités
})

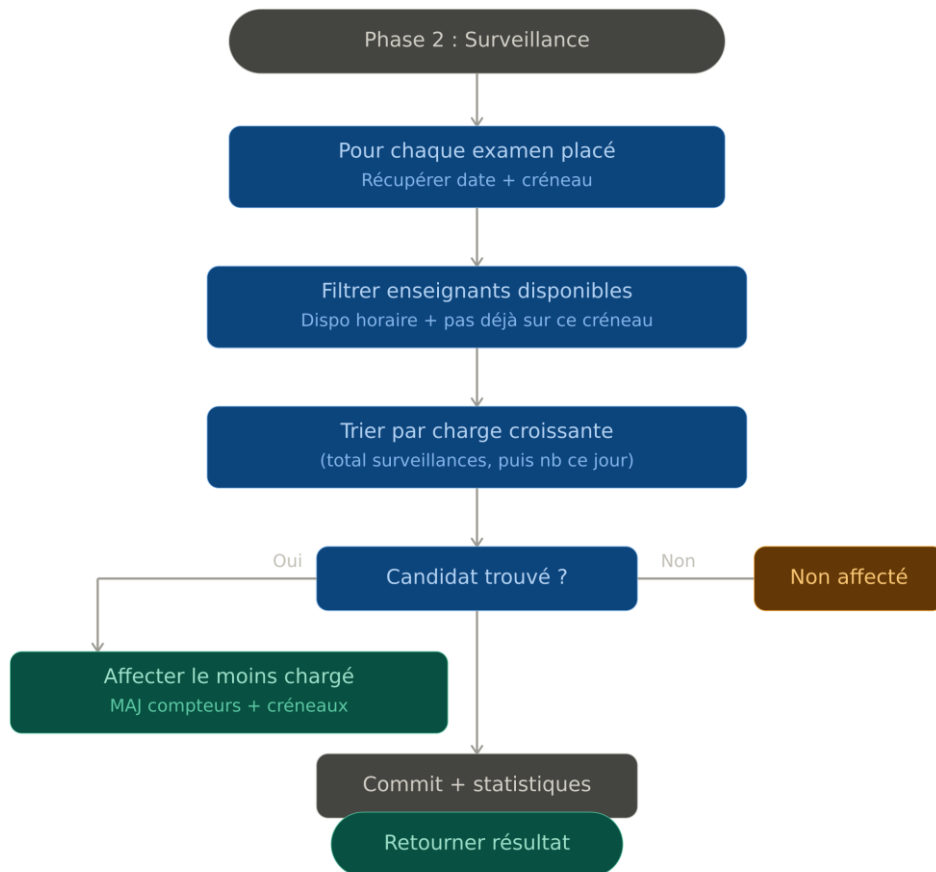
```

### Notes :

Où J=jours, M=matières/jour, S=slots/jour, K=salles, E=examens placés, T=enseignants.

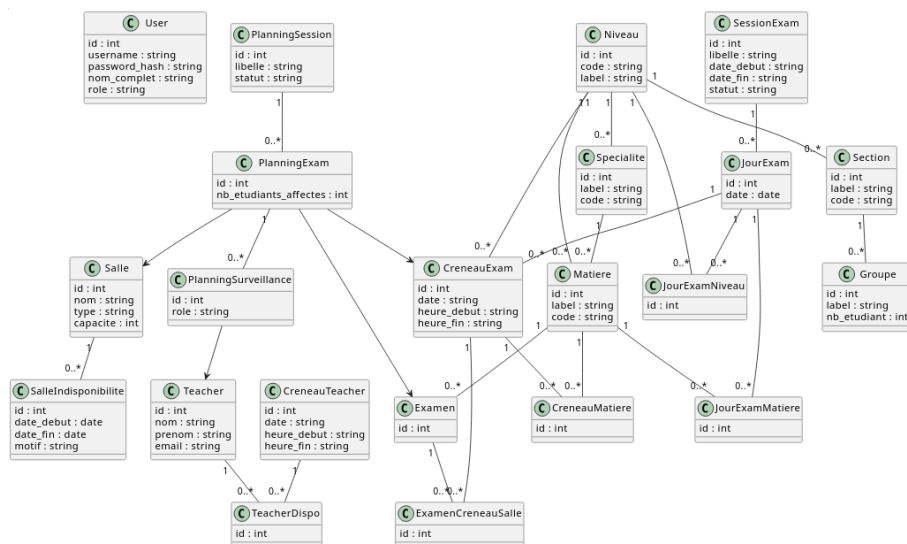
### Mind map de l'algo implementer :





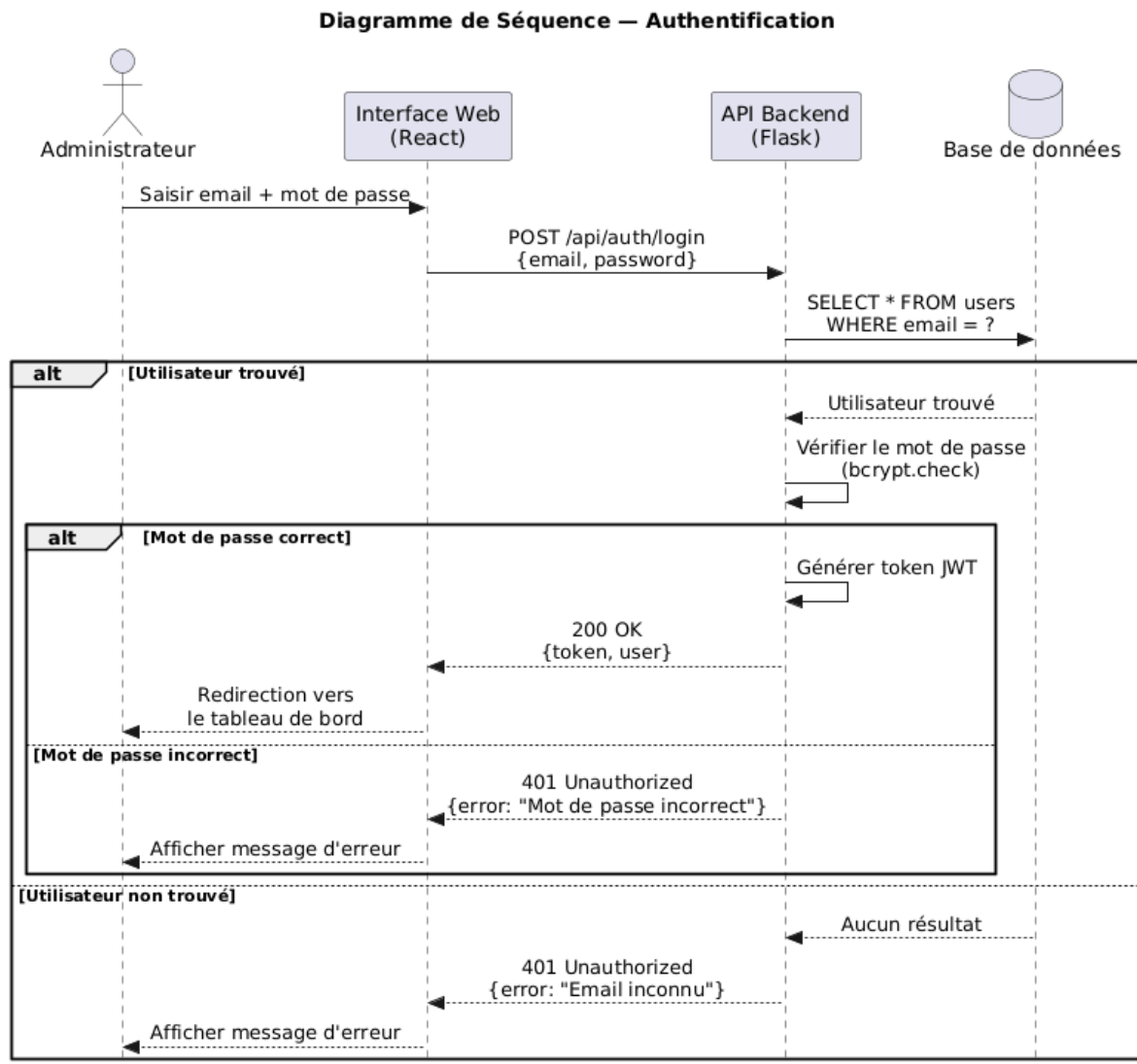
### 3.4 Conception détaillée (UML)

#### 3.4.1 Diagramme de classes

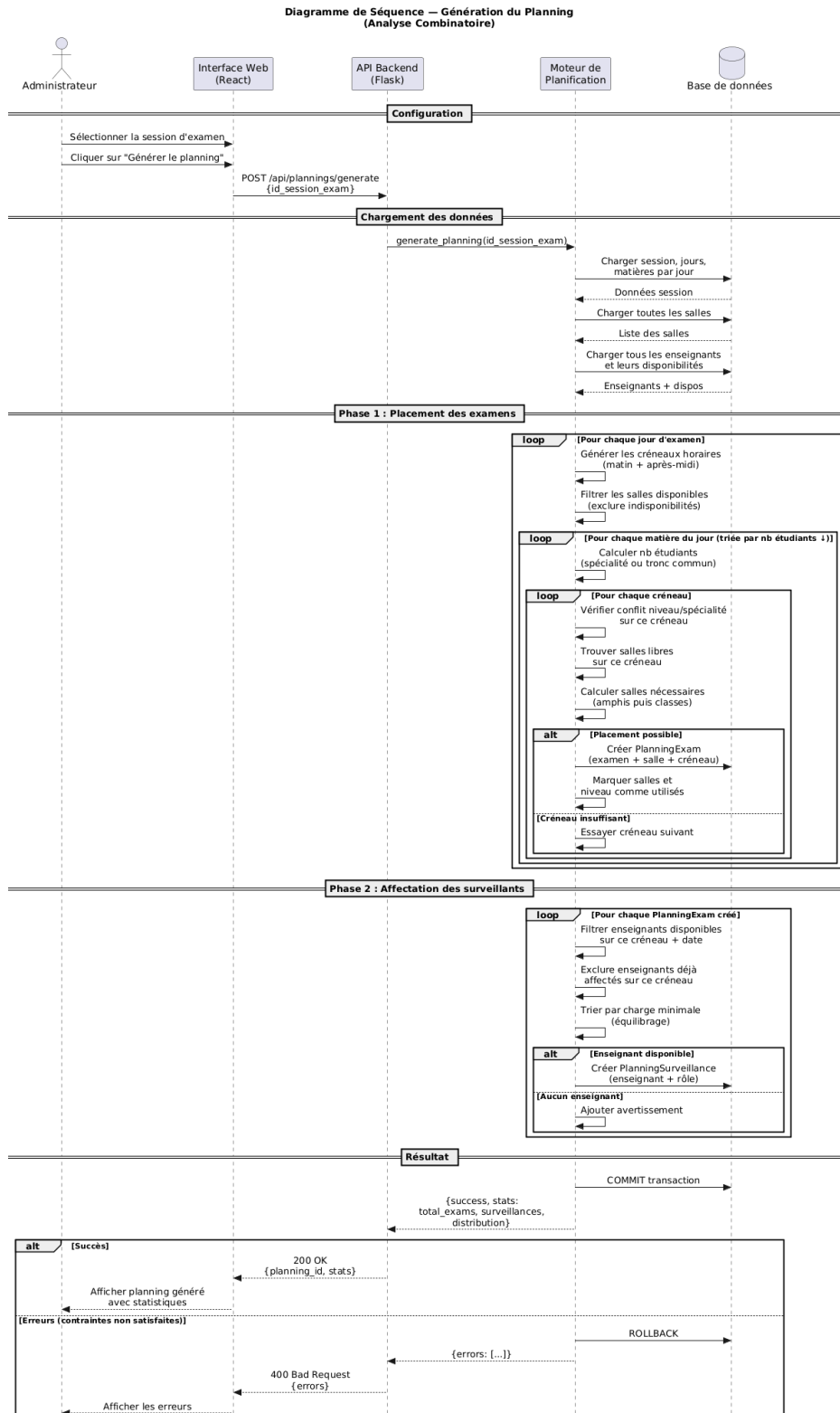


### 3.4.2 Diagramme de séquence

- Diagramme de séquence authentification



- Diagramme de séquence génération de planning



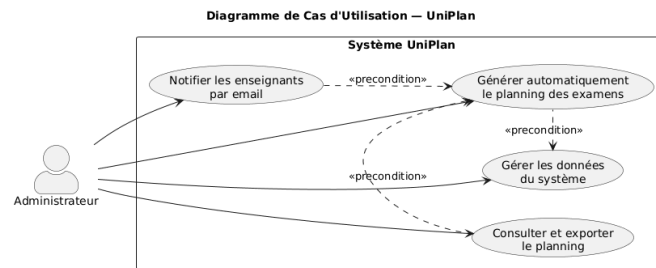
### 3.4.3 Diagramme de cas d'utilisation

Acteurs :

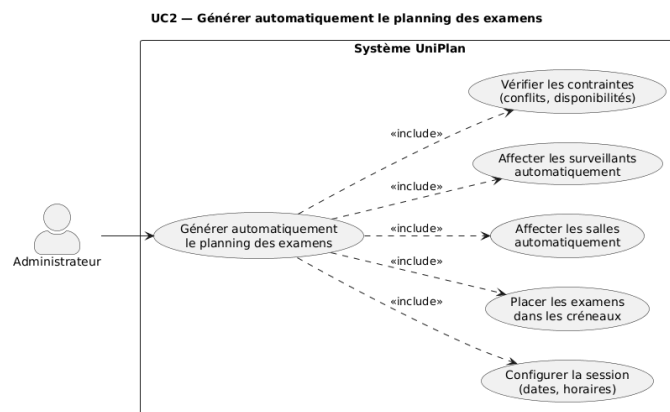
- Administrateur

Cas :

- Notifier des profs

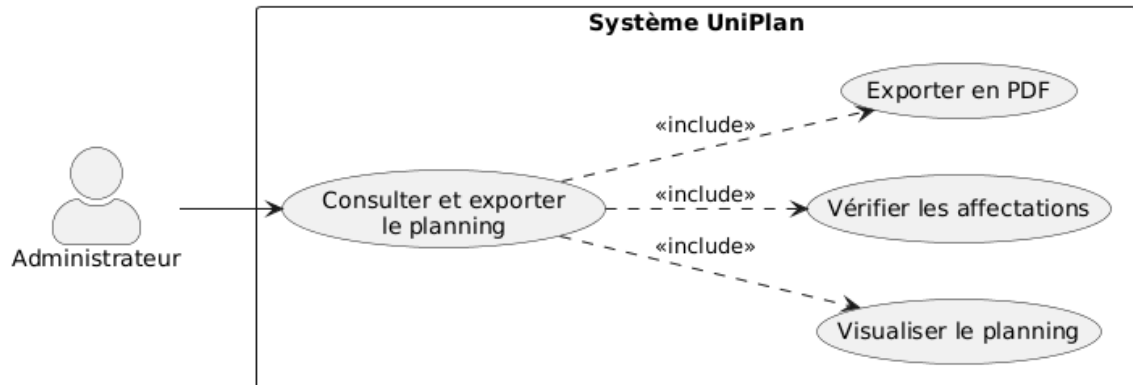


- Générer un examen

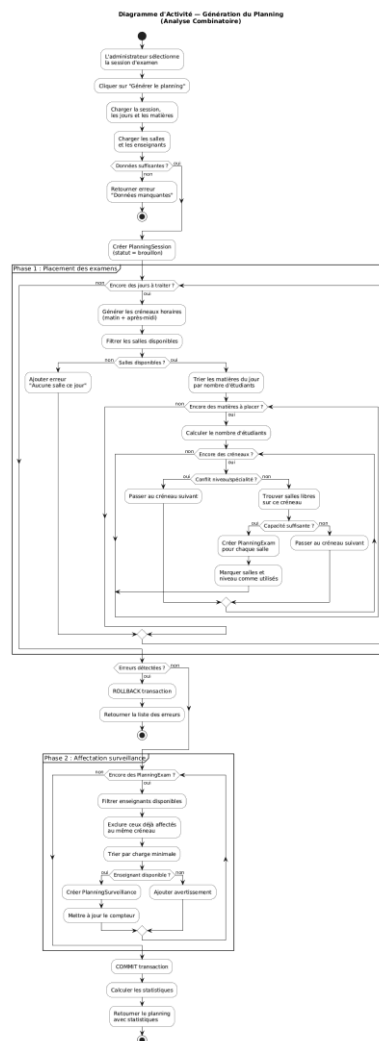


- Exporter un examen

### UC3 — Consulter et exporter le planning



### 3.4.4 Diagramme d'activité



### **3.5 Conclusion du chapitre**

La conception du système a permis de structurer clairement les différents composants et leurs interactions. L'architecture client-serveur adoptée assure une bonne séparation des responsabilités, facilitant ainsi la maintenance et l'évolution future du système.

La modélisation combinatoire permet de représenter efficacement le problème de planification, tandis que l'utilisation d'outils UML offre une vision claire et formelle du système.

Cette phase constitue une étape essentielle avant l'implémentation, qui sera abordée dans le chapitre suivant.

## **Chapitre 4 : Implémentation, Tests et Validation**

## 4.1 Introduction

Après les phases d'analyse et de conception, cette étape est consacrée à la réalisation concrète du système de planification des examens. Elle décrit les choix technologiques adoptés, l'environnement de développement, ainsi que l'architecture logicielle mise en place.

Ce chapitre présente également les interfaces utilisateur développées, les fonctionnalités implémentées, ainsi que les différentes stratégies de test utilisées pour valider le bon fonctionnement du système. L'objectif est de démontrer que la solution proposée est opérationnelle, fiable et conforme aux besoins définis précédemment.

## 4.2 Environnement de développement

### 4.2.1 Outils et langages

Le système a été développé en s'appuyant sur des technologies modernes et adaptées au développement d'applications web.

- **Langage de programmation :**
- Python 3.x, choisi pour sa simplicité, sa lisibilité et sa richesse en bibliothèques.
- **Framework web :**

Flask, un micro-framework léger permettant de développer rapidement des applications web modulaires et évolutives.

- **Base de données :**

PostgreSQL, utilisé pour le stockage des données relatives aux salles, enseignants, modules, étudiants et plannings.

Ce choix est motivé par :

- sa robustesse,
  - sa gestion avancée des relations,
  - sa performance sur les données complexes.
- **Front-end :**
  - HTML pour la structure
  - CSS pour le design
  - JavaScript pour l'interactivité

- Tailwind css pour améliorer l'ergonomie et le responsive design
- **Autres outils :**
  - SQLAlchemy (ORM) pour le mapping objet-relationnel
  - Git pour le versionnement du code

#### 4.2.2 Structure du code source

Le projet a été organisé de manière modulaire afin de faciliter la maintenance et l'évolution.

#### *Structure générale :*

|                                                                                                                  |                      |                                                                                               |
|------------------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------|
|  <b>smassaid</b> code completee | 90c02dc · 2 days ago |  5 Commits |
|  <b>__pycache__</b>             | code completee       | 2 days ago                                                                                    |
|  <b>models</b>                  | code completee       | 2 days ago                                                                                    |
|  <b>routes</b>                  | code completee       | 2 days ago                                                                                    |
|  <b>seeders</b>                | code completee       | 2 days ago                                                                                    |
|  <b>services</b>              | code completee       | 2 days ago                                                                                    |
|  <b>static</b>                | first commit         | 2 days ago                                                                                    |
|  <b>templates</b>             | first commit         | 2 days ago                                                                                    |
|  <b>venv</b>                  | first commit         | 2 days ago                                                                                    |
|  <b>README.md</b>             | Initial commit       | 2 days ago                                                                                    |
|  <b>app.py</b>                | first commit         | 2 days ago                                                                                    |
|  <b>extensions.py</b>         | first commit         | 2 days ago                                                                                    |
|  <b>requirements.txt</b>      | first commit         | 2 days ago                                                                                    |

#### 4.3 Réalisation et interfaces utilisateur

L'application propose une interface web intuitive permettant aux utilisateurs d'interagir facilement avec le système.

#### 4.4 Scénarios de tests et résultats

##### 4.4.1 Test des contraintes fortes

Afin de valider la robustesse du système, plusieurs scénarios de tests ont été réalisés.

### **1. Cas simple (petite instance)**

- Nombre réduit d'examens ( $\leq 10$ )
- Vérification manuelle des résultats

### **2. Cas réel (grande instance)**

- Simulation d'un planning réel
- Vérification automatique des contraintes

#### **Résultats :**

- Aucun conflit détecté entre examens
- Respect total des contraintes :
  - pas de double affectation
  - respect des capacités
  - respect des disponibilités

Cela confirme la validité de l'algorithme.

### **4.4.2 Évaluation des contraintes souples**

Les performances du système ont également été évaluées sur la base de critères d'optimisation.

#### **Indicateurs mesurés :**

- Nombre d'examens rapprochés pour un étudiant
- Répartition des examens sur les créneaux
- Équilibre des surveillances

#### **Résultats observés :**

- Réduction significative des conflits proches
- Meilleure répartition temporelle
- Charge plus équilibrée pour les enseignants

Comparé à une planification manuelle, le système offre une amélioration notable.

#### 4.4.3 Performance du système (AJOUT IMPORTANT)

- Temps de génération acceptable même pour des datasets importants
- Complexité maîtrisée grâce à :
  - pruning (élagage des solutions invalides)
  - simplification logique

#### 4.5 Conclusion du chapitre

L'implémentation du système en Python avec le framework Flask et la base de données PostgreSQL a permis de développer une application web fonctionnelle, robuste et évolutive.

Les tests réalisés ont démontré :

- la conformité du système aux contraintes fortes,
- l'efficacité de l'approche combinatoire,
- et une amélioration significative de la qualité des plannings générés.

Cette phase valide ainsi la pertinence de la solution proposée et confirme sa capacité à répondre aux besoins réels d'un établissement universitaire.

### Conclusion Générale et Perspectives

#### Conclusion générale

Ce projet a permis de concevoir et de développer une application web dédiée à la planification des examens de session, en s'appuyant sur une approche combinatoire et une implémentation en Python/Flask avec une base de données PostgreSQL.

Le système proposé permet :

- d'automatiser la génération des plannings,
- de garantir le respect des contraintes fortes,
- d'améliorer l'équité grâce aux contraintes souples,
- et de faciliter le travail des responsables de scolarité.

L'intégration d'une interface web intuitive rend l'outil accessible et exploitable dans un contexte réel.

## Perspectives d'amélioration

Malgré les résultats satisfaisants obtenus, plusieurs axes d'amélioration peuvent être envisagés :

### ◇ Optimisation avancée

- Intégration de métaheuristiques :
  - algorithmes génétiques
  - recuit simulé

### ◇ Optimisation multi-objectifs

- Minimiser simultanément :
  - conflits proches
  - nombre de créneaux
  - charge enseignants

### ◇ Extension du système

- Version multi-établissement
- API REST pour intégration avec d'autres systèmes

### ◇ Amélioration de l'interface

- Calendrier interactif
- Visualisations graphiques (heatmaps, stats)

### ◇ Intelligence artificielle (bonus ++)

- prédiction des conflits
- suggestion automatique de planning optimal

## Ouverture

Ce projet constitue une base solide pour le développement d'un système d'aide à la décision dans le domaine de la gestion académique. Il peut être étendu et adapté à

d'autres problèmes de planification complexes, tels que les emplois du temps ou la gestion des ressources pédagogiques.